

NotebookLM Source: Template Completo de Código (Arquitetura SQLite + Event Log)

Este documento serve como referência de código-fonte funcional para todas as camadas que devem ser implementadas na avaliação utilizando o padrão SQLite puro da Semana 15.

1. domain.py (Modelo de Domínio)

Lógica de negócio pura, entidades, objetos de valor e agregação.

```
from dataclasses import dataclass
from typing import List, Optional

@dataclass(frozen=True)
class OrderLine:
    orderid: str
    sku: str
    qty: int

class Batch:
    def __init__(self, ref: str, sku: str, qty: int):
        self.ref = ref
        self.sku = sku
        self.qty = qty
        self.allocations: set[OrderLine] = set()

    @property
    def allocated_quantity(self) -> int:
        return sum(line.qty for line in self.allocations)

    @property
    def available_quantity(self) -> int:
        return self.qty - self.allocated_quantity

    def can_allocate(self, line: OrderLine) -> bool:
        return self.sku == line.sku and self.available_quantity >= line.qty

    def allocate(self, line: OrderLine) -> bool:
        if self.can_allocate(line):
            self.allocations.add(line)
            return True
        return False

class Product:
    def __init__(self, sku: str):
        self.sku = sku
        self.batches: List[Batch] = []
        self.events: List = [] # Rastreamento de eventos de domínio

    def add_batch(self, batch: Batch):
        self.batches.append(batch)

    def allocate(self, line: OrderLine) -> Optional[str]:
        # Tenta alocar no primeiro lote disponível
        for batch in self.batches:
            if batch.allocate(line):
                # Importante: O domínio não publica eventos diretamente no banco,
                # apenas os armazena na lista interna para processamento posterior.
                from messages import Allocated
                self.events.append(
```

```

        Allocated(
            orderid=line.orderid,
            sku=line.sku,
            qty=line.qty,
            batchref=batch.ref,
        )
    )
    return batch.ref

# Caso falhe a alocação em todos os lotes
from messages import OutOfStock
self.events.append(OutOfStock(sku=line.sku))
return None

```

2. messages.py (Comandos e Eventos)

Definições dos sinais de dados que circulam na aplicação.

```

from dataclasses import dataclass

class Command:
    pass

class Event:
    pass

# --- COMANDOS (Ações imperativas) ---
@dataclass
class CreateBatch(Command):
    ref: str
    sku: str
    qty: int

@dataclass
class Allocate(Command):
    orderid: str
    sku: str
    qty: int

# --- EVENTOS (Fatos históricos) ---
@dataclass
class Allocated(Event):
    orderid: str
    sku: str
    qty: int
    batchref: str

@dataclass
class OutOfStock(Event):
    sku: str

```

3. repository.py (Repositório SQLite)

Camada que acessa o banco usando consultas SQLite puras (sem SQLAlchemy).

```

from domain import Product, Batch, OrderLine

class SQLiteRepository:
    def __init__(self, conn):

```

```

        self.conn = conn

    def add_batch(self, ref: str, sku: str, qty: int):
        self.conn.execute(
            "INSERT INTO batches (ref, sku, qty) VALUES (?, ?, ?)",
            (ref, sku, qty),
        )

    def get_product(self, sku: str) -> Optional[Product]:
        # 1. Busca todos os lotes correspondentes ao SKU
        rows = self.conn.execute(
            "SELECT ref, sku, qty FROM batches WHERE sku = ?",
            (sku,),
        ).fetchall()

        if not rows:
            return None

        # 2. Reconstrói o agregado Product do domínio
        product = Product(sku)

        for row in rows:
            batch = Batch(row["ref"], row["sku"], row["qty"])

            # 3. Busca e adiciona as alocações físicas de cada lote
            allocation_rows = self.conn.execute(
                "SELECT orderid, sku, qty FROM allocations WHERE batchref = ?",
                (row["ref"],),
            ).fetchall()

            for allocation in allocation_rows:
                batch.allocations.add(
                    OrderLine(
                        orderid=allocation["orderid"],
                        sku=allocation["sku"],
                        qty=allocation["qty"],
                    )
                )

            product.add_batch(batch)

        return product

    def save_allocation(self, orderid: str, sku: str, qty: int, batchref: str):
        self.conn.execute(
            "INSERT INTO allocations (orderid, sku, qty, batchref) VALUES (?, ?, ?, ?)",
            (orderid, sku, qty, batchref),
        )

```

4. database.py (Conexão e Criação de Tabelas)

Responsável pelo gerenciamento de conexão SQLite3 e geração de tabelas.

```

import sqlite3

DB_NAME = "estoque.db"

def get_connection():
    conn = sqlite3.connect(DB_NAME)
    # Define a row_factory para permitir acessar colunas por nome: row["coluna"]
    conn.row_factory = sqlite3.Row
    return conn

def create_tables():

```

```

conn = get_connection()

# Tabela de lotes
conn.execute("""
CREATE TABLE IF NOT EXISTS batches (
    ref TEXT PRIMARY KEY,
    sku TEXT NOT NULL,
    qty INTEGER NOT NULL
)
""")

# Tabela de alocações
conn.execute("""
CREATE TABLE IF NOT EXISTS allocations (
    orderid TEXT PRIMARY KEY,
    sku TEXT NOT NULL,
    qty INTEGER NOT NULL,
    batchref TEXT NOT NULL
)
""")

# Tabela de log de eventos (Transactional Outbox)
conn.execute("""
CREATE TABLE IF NOT EXISTS event_log (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    channel TEXT NOT NULL,
    event_type TEXT NOT NULL,
    payload TEXT NOT NULL
)
""")

conn.commit()
conn.close()

```

5. handlers.py (Casos de Uso)

Contém a lógica das ações do sistema, interagindo com o UOW e publicando eventos.

```

from domain import OrderLine
from messages import Allocated, OutOfStock
from publisher import publish_event

def create_batch_handler(command, uow):
    with uow:
        # Chama a persistência no repositório diretamente
        uow.products.add_batch(command.ref, command.sku, command.qty)
        uow.commit()

def allocate_handler(command, uow) -> Optional[str]:
    with uow:
        # Busca o agregado do banco
        product = uow.products.get_product(command.sku)

        if product is None:
            raise ValueError(f"SKU inválido: {command.sku}")

        # Tenta alocar utilizando a lógica do domínio
        line = OrderLine(command.orderid, command.sku, command.qty)
        batchref = product.allocate(line)

        # Se não houver lote disponível com estoque
        if batchref is None:
            publish_event("out_of_stock", OutOfStock(command.sku))
            return None

```

```

# Salva a alocação no banco
uow.products.save_allocation(
    command.orderid,
    command.sku,
    command.qty,
    batchref,
)
uow.commit()

# Publica o evento de sucesso no log de eventos
publish_event(
    "line_allocated",
    Allocated(
        command.orderid,
        command.sku,
        command.qty,
        batchref,
    ),
)

return batchref

```

6. read_model.py (Modelo de Leitura / CQRS)

Consultas diretas e de alta performance no banco de dados para rotas de GET.

```

from database import get_connection

def list_allocations() -> list[dict]:
    conn = get_connection()

    rows = conn.execute(
        "SELECT orderid, sku, qty, batchref FROM allocations ORDER BY orderid"
    ).fetchall()

    conn.close()

    # Converte rows do sqlite3.Row em dicionários comuns
    return [dict(row) for row in rows]

```